

ACTION PLAN • v1.0

NumanOS

Personal Automation Platform

A unified, secure, AI-augmented personal CRM and portfolio that automates everything currently done by hand — expenses, files, notes, client meetings, emails and projects.

Frontend React + Vite + TS	Backend Python Flask	Database PostgreSQL (managed)	AI LangChain + RAG
--------------------------------------	--------------------------------	---	------------------------------

Prepared for **Numan Iftikhar** — Senior Multi-Cloud DevOps Engineer

Date: 21 May 2026

Profile: numaniftikhar.com • Design reference: ericwadkins.com

SECTION 1

Executive Summary

This document is the build plan for **NumanOS** — a single private platform that turns your manual day-to-day workflows into automated, auditable software. Today you track expenses, store files, write notes, book client meetings, draft emails and manage projects across scattered tools. NumanOS folds all of that into one secure web application with a public portfolio front-end, backed by a proper cloud PostgreSQL database and an AI layer (LangChain + Retrieval-Augmented Generation) that can answer questions over your own data and draft your emails.

The stack is fixed per your direction: a **React** single-page front-end, a **Python Flask** REST API, and a **managed PostgreSQL** database. The architecture is cloud-native and container-first so it deploys cleanly to any of AKS / EKS / GKE — squarely in your wheelhouse as a Kubernetes and IaC specialist. Delivery is split into **8 phases** so a usable product ships early (auth + portfolio + meetings) and the heavier AI and automation features layer in afterward.

10 Feature pillars	8 Delivery phases	3 Tier architecture	~16–22 wk Solo build estimate
-----------------------	----------------------	------------------------	----------------------------------

Guiding principle

Automate the manual, secure by default, and ship in thin vertical slices. Every phase ends with something you can actually log into and use — not a half-built layer.

SECTION 2

Profile — Source Content for the Portfolio

Pulled from numaniftikhar.com to seed the public portfolio module. You will be able to edit all of this from the admin UI; nothing is hard-coded.

Field	Value
Name / Title	Numan Iftikhar — Senior Multi-Cloud DevOps Engineer
Positioning	Kubernetes Architect & IaC Expert; turns manual workflows into GitOps-driven automation
Experience	5+ years across Azure, AWS and GCP; current: Sr. DevOps @ TrueMedIT (2023–present)
Core stack	Kubernetes (AKS/EKS/GKE), Docker, Terraform, Ansible, ArgoCD, Helm, KEDA, Istio
CI/CD & Sec	GitHub Actions, GitLab CI, Jenkins, Cloud Build; Vault, OPA, Trivy, Cosign, Prisma
Observability	Prometheus, Grafana, Cloud Monitoring
Certifications	8 total — GCP DevOps/Architect/ACE, AWS SAA + DVA, Azure Admin, Terraform Assoc.
Flagship work	FinGuard (multi-cloud banking, PCI-DSS/SOC 2); Lab Mgmt SaaS on AKS (HIPAA, 99.97%)
Contact	hellonumaniftikhar@gmail.com • linkedin.com/in/numaniftikhar • github.com/numaniftikhar1088

SECTION 3

System Architecture

NumanOS is a classic three-tier app hardened for cloud: a React SPA talks to a Flask REST API over HTTPS/JWT; the API owns all business logic and is the only thing that touches PostgreSQL, object storage and the AI services. Long-running work (document indexing, email drafting, file processing) is pushed to a background worker so the API stays responsive.

CLIENT — React SPA (Vite + TS, Tailwind/shadcn): Portfolio · Dashboard · Notes · Files · Expenses · AI Chat
EDGE — HTTPS · JWT auth + MFA gate · rate limiting · CORS · WAF / reverse proxy (Nginx / Ingress)
API — Flask (blueprints) · SQLAlchemy · Marshmallow validation · service layer · OpenAPI
ASYNC — Celery workers + Redis broker: RAG indexing · email drafting · OCR · file scans · reminders
DATA — PostgreSQL (+ pgvector for embeddings) Object storage (S3 / GCS / Blob) Redis cache
AI — LangChain orchestration · embeddings · LLM (Claude / configurable) · retrieval over pgvector
PLATFORM — Docker · Kubernetes (AKS/EKS/GKE) · Terraform IaC · GitHub Actions CI/CD · Prometheus + Grafana

Why pgvector instead of a separate vector DB

Storing embeddings in PostgreSQL via the pgvector extension keeps the entire system on one managed database — one backup story, one access-control model, one bill. It satisfies your 'proper single cloud database' requirement and removes a moving part (no Pinecone/Weaviate to run).

SECTION 4

Technology Stack

Layer	Choice	Notes
Frontend	React 18 + Vite + TypeScript	Fast builds, typed; SPA with code-splitting
UI / styling	Tailwind CSS + shadcn/ui	Modern, accessible components; dark-mode native
Client state	TanStack Query + Zustand	Server cache + light global state
Routing	React Router	Public portfolio + protected app routes
Backend	Flask + Blueprints	REST API; per-feature modules
ORM / migrations	SQLAlchemy + Alembic	Versioned schema changes
Validation / docs	Marshmallow + apispec (OpenAPI)	Request schemas + auto Swagger UI
Async	Celery + Redis	Background jobs and scheduled tasks (beat)
Database	PostgreSQL + pgvector	Managed: Cloud SQL / RDS / Azure DB
Object storage	S3 / GCS / Azure Blob	Personal cloud storage + receipts/attachments
Auth	JWT (access+refresh) + pyotp TOTP	Argon2 hashing; MFA + backup codes
AI	LangChain + LLM + embeddings	RAG over personal data; email drafting
Infra	Docker + Kubernetes + Terraform	CI/CD via GitHub Actions; Prometheus/Grafana

SECTION 5

Feature Breakdown — The 10 Pillars

Each pillar maps to a backend module, a set of React screens and its own tables. Below: scope, key tech and the core data each owns.

Pillar / Scope	Owns (data)
1 • Expense Tracker Transactions, categories, budgets and recurring expenses with charts and CSV import/export. Receipt upload to object storage with optional OCR auto-fill. Monthly dashboards and alerts.	Tables: accounts, transactions, categories, budgets, receipts.
2 • Personal Cloud Storage Upload / download / preview, nested folders, versioning, share links with expiry, and per-account quota. Files in object storage; metadata + permissions in PostgreSQL; encrypted at rest.	Tables: files, folders, file_versions, share_links.
3 • Portfolio + Client Meetings Public, editable portfolio (hero, about, projects, certs, services) plus client booking: availability rules, time-slot picker, auto Google Meet/Zoom link, email + calendar invite, reminders.	Tables: profile, portfolio_sections, availability, bookings.
4 • MFA / Security TOTP via authenticator apps (pyotp), one-time backup codes, optional email OTP, enforced at login with a recovery flow. Argon2 password hashing; refresh-token rotation.	Tables: users, mfa_secrets, backup_codes, sessions.
5 • Cloud Database Foundation Managed PostgreSQL with Alembic migrations, automated backups + PITR, connection pooling (PgBouncer), and pgvector enabled for the AI layer. Read replica optional later.	Cross-cutting: powers every other module.
6 • RAG + LangChain Ingest notes, files and emails -> chunk -> embed -> store in pgvector. A LangChain retrieval chain answers questions over your own knowledge base with citations back to the source document.	Tables: documents, chunks, embeddings (vector), chat_threads.
7 • Email Generation LLM-drafted emails (client follow-ups, cold outreach, meeting confirmations) with reusable templates and tone control; review-then-send via Gmail API; full sent-history log.	Tables: email_templates, drafts, sent_emails.
8 • Notes (Notion-style) Block-based rich-text editor (TipTap / BlockNote), nested pages, tags, full-text search and markdown. Notes feed straight into the RAG index so the AI can reason over them.	Tables: notes, blocks (JSONB), note_tags.
9 • Service Integrations OAuth connection framework for Gmail, Google Calendar, Google Drive, Calendly, Slack, GitHub and Stripe (invoicing). Each integration is a pluggable module with stored, scoped tokens.	Tables: integrations, oauth_tokens, webhooks.
10 • Projects / 'Now' Kanban board of projects with status, tech stack, links and notes; the 'what I'm working on now' view publishes selected items straight to the public portfolio.	Tables: projects, project_tasks, project_links.

SECTION 6

Data Model — Core Entities

High-level relational map. A single **users** row anchors everything; the AI layer reuses the same database via pgvector rather than a separate store.

Domain	Primary tables	Key relationships
Identity	users, sessions, mfa_secrets, backup_codes	user 1—* sessions; user 1—1 mfa_secret
Expenses	accounts, transactions, categories, budgets, receipts	account 1—* transactions; txn *—1 category
Storage	folders, files, file_versions, share_links	folder 1—* files; file 1—* versions
Portfolio	profile, portfolio_sections, availability, bookings	profile 1—* sections; availability -> bookings
Notes	notes, blocks, note_tags	note 1—* blocks; note *—* tags
AI / RAG	documents, chunks, embeddings, chat_threads, messages	document 1—* chunks; chunk 1—1 embedding(vector)
Email	email_templates, drafts, sent_emails	template 1—* drafts; draft 1—1 sent_email
Integrations	integrations, oauth_tokens, webhooks	integration 1—* tokens
Projects	projects, project_tasks, project_links	project 1—* tasks / links

Migrations & integrity

Every schema change ships as an Alembic migration in version control. Foreign keys, NOT NULL and CHECK constraints are enforced at the DB level — not just in app code — so data stays consistent even under concurrent background jobs.

SECTION 7

Security & MFA

- **Authentication:** email + password (Argon2id), then mandatory TOTP MFA. Short-lived JWT access tokens with rotating refresh tokens; sessions revocable from the UI.
- **MFA flows:** enrol via QR code (pyotp), 10 single-use backup codes, optional email OTP fallback, and a documented account-recovery path.
- **Authorization:** single-owner model now, with a role/permission table ready for future shared access.
- **Data protection:** TLS everywhere; secrets in a vault (HashiCorp Vault / cloud secrets manager — your toolset); object storage and DB encrypted at rest; signed, expiring URLs for file access.
- **App hardening:** input validation (Marshmallow), parameterised queries (SQLAlchemy), rate limiting, CORS allow-list, security headers, audit log of sensitive actions.
- **Pipeline security:** Trivy image scans, dependency scanning, and SBOM/Cosign signing in CI — reusing the DevSecOps stack from your portfolio.

SECTION 8

AI Layer — RAG, LangChain & Email Generation

The AI layer is what makes NumanOS feel automated rather than just a database with forms. It runs as services behind the Flask API and does its heavy lifting in Celery workers.

Retrieval-Augmented Generation pipeline

- **Ingest:** notes, uploaded files (PDF/DOCX/TXT) and selected emails are normalised to text.
- **Chunk + embed:** text is split into overlapping chunks and embedded; vectors land in pgvector.
- **Retrieve + answer:** a LangChain retrieval chain pulls the most relevant chunks and the LLM answers with inline citations linking back to the source note or file.
- **Chat:** threaded conversations stored in PostgreSQL; ask things like 'summarise my Q2 expenses' or 'what did I note about the FinGuard failover design?'

Email generation

- Pick a template + intent (follow-up, proposal, meeting confirm) and the LLM drafts a tone-matched email.
- RAG can inject real context (the client's project notes, last meeting summary) into the draft.
- Always review-then-send; sending goes through the Gmail integration and is logged.

LLM provider — recommendation

Default to Anthropic Claude (latest Sonnet/Opus) for drafting and RAG answers, with the provider kept behind a thin LangChain interface so it stays swappable (OpenAI, or a self-hosted open model on your own GPU/K8s if you prefer full data residency). This is a decision to confirm — see Section 12.

SECTION 9

Repository & Project Structure

A monorepo keeps frontend, backend and infrastructure versioned together for clean CI/CD.

```
numanos/
├── frontend/                # React + Vite + TS SPA
│   ├── src/{components,pages,features,hooks,lib,api}
│   └── Dockerfile
├── backend/                 # Flask API
│   ├── app/
│   │   ├── modules/        # expenses, storage, notes, ai, email, ...
│   │   ├── core/           # auth, mfa, config, db, security
│   │   ├── workers/        # celery tasks
│   │   └── extensions.py
│   ├── migrations/         # alembic
│   ├── tests/
│   └── Dockerfile
├── infra/                   # terraform (db, storage, k8s, secrets)
├── deploy/                  # helm charts / k8s manifests
├── .github/workflows/       # CI: lint, test, scan, build, deploy
└── docker-compose.yml       # local dev (api, db, redis, worker, web)
```

SECTION 10

Delivery Roadmap — 8 Phases

Vertical slices: each phase ends with something usable. Estimates assume a focused solo build; they compress significantly with help.

Phase	Focus	Key deliverables	Est.
0	Foundations	Monorepo, Docker compose, CI skeleton, base Flask+React, Postgres connected	1 wk
1	Auth + MFA	Signup/login, JWT, TOTP MFA, backup codes, sessions, user profile	1–2 wk
2	Portfolio + Meetings	Public portfolio (editable), projects, booking + calendar invites	2–3 wk
3	Notes	Block editor, nested pages, tags, search	2 wk
4	Cloud Storage	Upload/download, folders, versioning, share links, quotas	2 wk
5	Expense Tracker	Transactions, budgets, receipts, dashboards, CSV	2 wk
6	AI Layer	pgvector RAG, LangChain chat, email generation	2–3 wk
7	Integrations + Hardening	Gmail/Calendar/Drive/Slack/GitHub, observability, IaC deploy	2–3 wk

Why this order

Auth + portfolio + meetings go first so you get a real, public, client-facing tool live within weeks. Notes and storage build the content that the AI layer (Phase 6) then indexes — so RAG has something to retrieve the moment it ships.

SECTION 11

DevOps, Deployment & Infrastructure

This section leans on your own specialty — the platform is designed to be deployed the way you already build for clients.

- **Containers:** each service (api, worker, web) ships as a minimal multi-stage Docker image.
- **Orchestration:** Kubernetes (AKS/EKS/GKE) via Helm; HPA/KEDA for the worker, Ingress + cert-manager for TLS.
- **IaC:** Terraform provisions the managed Postgres, object storage, secrets, DNS and the cluster — reproducible across environments.
- **CI/CD:** GitHub Actions — lint, test, Trivy scan, build, sign (Cosign), push, then GitOps deploy (ArgoCD).
- **Observability:** Prometheus metrics + Grafana dashboards + structured logs; uptime and error alerting.
- **Environments:** local (docker-compose) -> staging -> production, with automated DB backups + PITR.

Lean start option

For a single-user app you don't need a full cluster on day one. A managed Postgres + a single container host (Cloud Run / App Runner / a small VM) runs NumanOS cheaply, and the Terraform + Helm setup means you can graduate to full Kubernetes whenever you want — no rewrite.

SECTION 12

Indicative Monthly Running Cost

Rough, single-user, production-grade estimate (USD). Actuals depend on cloud, region and usage.

Component	Lean setup	Scaled / K8s setup
Managed PostgreSQL (+pgvector)	\$15–30	\$50–120
Object storage + egress	\$2–10	\$10–40
Compute (API + worker)	\$10–25 (Cloud Run / small VM)	\$70–200 (K8s nodes)
Redis (cache + broker)	\$0–10	\$15–40
LLM + embeddings (API usage)	\$5–30 (usage-based)	\$30–150
Domain + TLS + email sending	\$2–10	\$5–20
Estimated total / month	≈ \$35–115	≈ \$180–570

Self-hosting the LLM on your own GPU/K8s removes per-token cost but adds compute. Start lean; the architecture scales without a rewrite.

SECTION 13

Risks & Mitigations

Risk	Impact	Mitigation
Scope is very large (10 pillars)	Burnout / stalls	Strict phased slices; each phase shippable on its own
AI cost / quality drift	Cost + UX	Provider behind interface; cap usage; cache embeddings
Sensitive data (expenses, files)	Privacy/security	MFA, encryption, vault secrets, audit log, least privilege
Third-party API limits/changes	Integration breakage	Pluggable integrations, retries, webhook verification
Solo maintenance load	Long-term upkeep	IaC + CI/CD + tests so it is reproducible and safe to change

SECTION 14

Open Decisions & Immediate Next Steps

Decisions to confirm before Phase 0

- **Cloud provider** — GCP (fits your Pro Cloud Architect cert), AWS, or Azure? Drives Terraform + managed services.
- **LLM provider** — Anthropic Claude (recommended), OpenAI, or self-hosted open model?
- **Deployment target** — start lean (Cloud Run / small VM) or go straight to Kubernetes?
- **Brand / name** — keep working title 'NumanOS' or use your own product name and domain?
- **Auth scope** — single-user only, or design now for future client logins / multi-user?

Immediate next steps (on approval)

- Confirm the decisions above (I can also recommend defaults and just proceed).
- Scaffold the monorepo: Flask API + React app + docker-compose + Postgres, running locally end-to-end.
- Build Phase 1 (Auth + MFA) as the first vertical slice you can log into.
- Stand up the Terraform + CI skeleton in parallel so deploys work from day one.

Ready when you are

Approve this plan (or tell me your picks for the open decisions) and I'll start with the monorepo scaffold and Phase 1. I'll keep delivering one usable slice at a time.